CS-218 DATA STRUCTURES AND ALGORITHMS

LECTURE 26

BY UROOJ AINUDDIN

GRAPHS

BOOK 4 CHAPTER 8

IN THE LAST LECTURE…

We found the weight matrix for a weighted digraph.

We executed the SPA on a weighted digraph.

We investigated the time and space complexities of SPA.

GRAPH TRAVERSAL

Algorithms and data structures

SEARCH ALGORITHMS

- The sequence generated by graph traversal greatly depends on the starting node.
- Traversal is mostly done to find a path from a source node S to a destination node D, if such a path exists.
- There are two algorithms that help traverse graphs:
  1. Depth-first search (DFS),
  2. Breadth-first search (BFS).

BREADTH-FIRST SEARCH (BFS)

- This traversal technique uses a queue.
- Each node has a STATUS variable.
- If STATUS is 1, the node has not been added to the queue.
- If STATUS is 2, the node has been added to the queue, where it is waiting to be visited.
- If STATUS is 3, the node has been visited.

BFS ALGORITHM

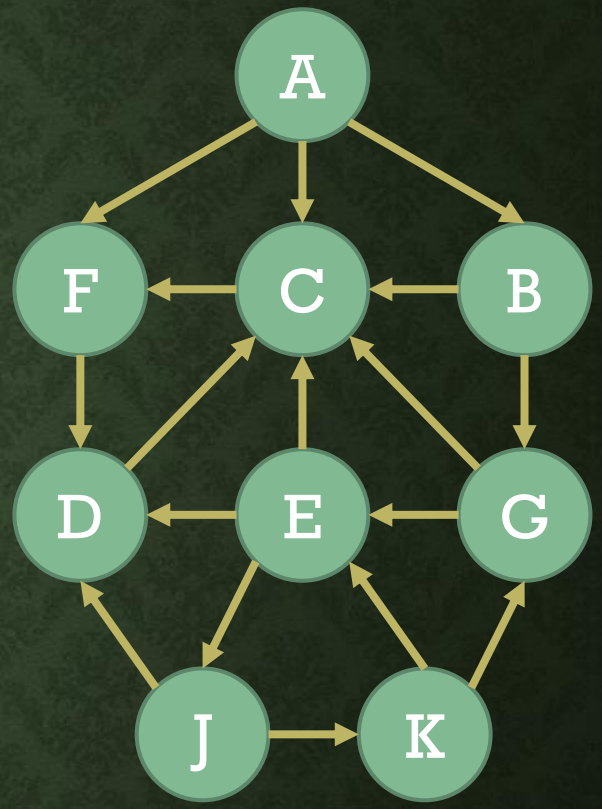
- Input: Graph G stored in memory, starting node X.

1. Set STATUS=1 for all nodes.
2. Put X in queue Q. Set STATUS=2 for X.
3. Repeat until Q is empty:
   a. Dequeue from Q. Let this node be N. Visit N. Set STATUS=3 for N.
   b. Enqueue all neighbors of N whose STATUS is 1 and set STATUS=2 for them.
4. Exit.

FIND A PATH FROM A TO J

- We maintain the queue Q.
- In addition, we maintain a list O, which keeps the name of the node which led to the insertion of this node in Q.

Node	Neighbors	STATUS
A	B, C, F	1
B	C, G	1
C	F	1
D	C	1
E	C, D, J	1
F	D	1
G	C, E	1
J	D, K	1
K	E, G	1



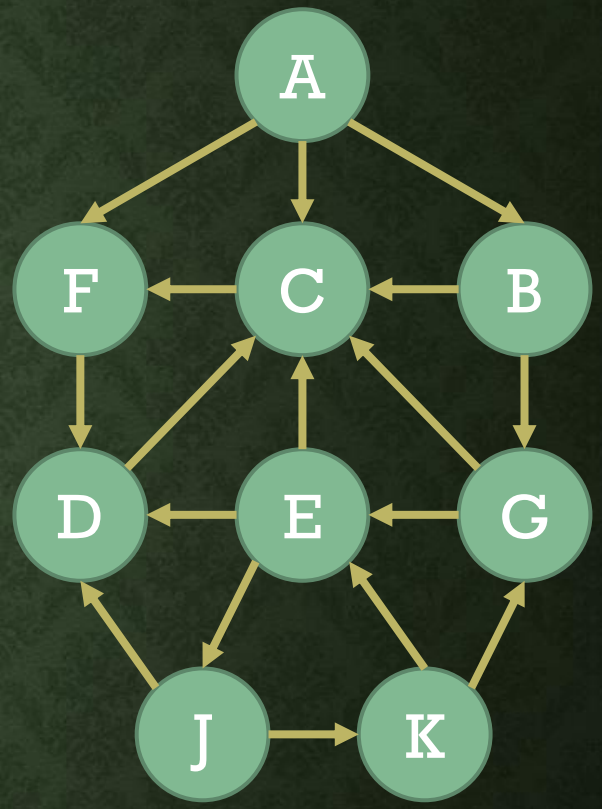
Q→ F=×, R=×

O→

FIND A PATH FROM A TO J

- We enqueue the source node.

Node	Neighbors	STATUS
A	B, C, F	2
B	C, G	1
C	F	1
D	C	1
E	C, D, J	1
F	D	1
G	C, E	1
J	D, K	1
K	E, G	1



0

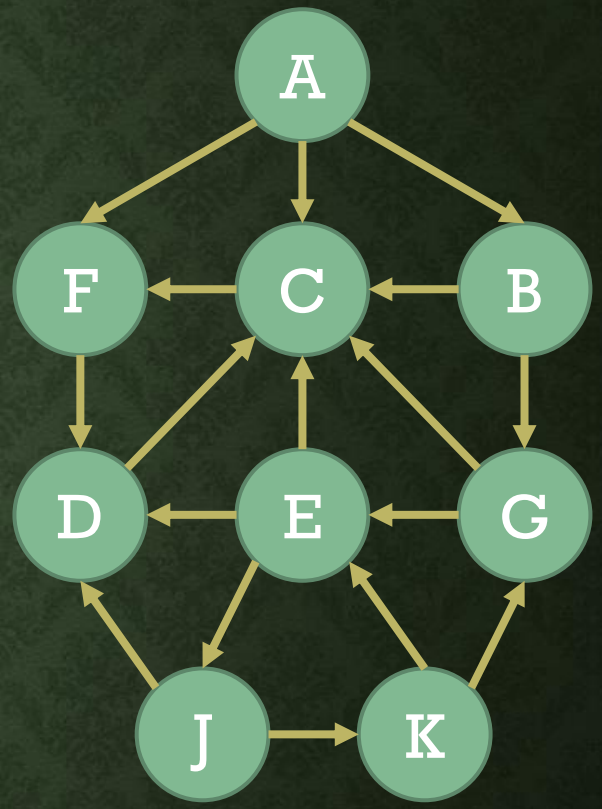
Q→ A F=0, R=0

O→ ∅

FIND A PATH FROM A TO J

- We visit the node at the head of the queue after dequeuing it.

Node	Neighbors	STATUS
A	B, C, F	3
B	C, G	1
C	F	1
D	C	1
E	C, D, J	1
F	D	1
G	C, E	1
J	D, K	1
K	E, G	1



0

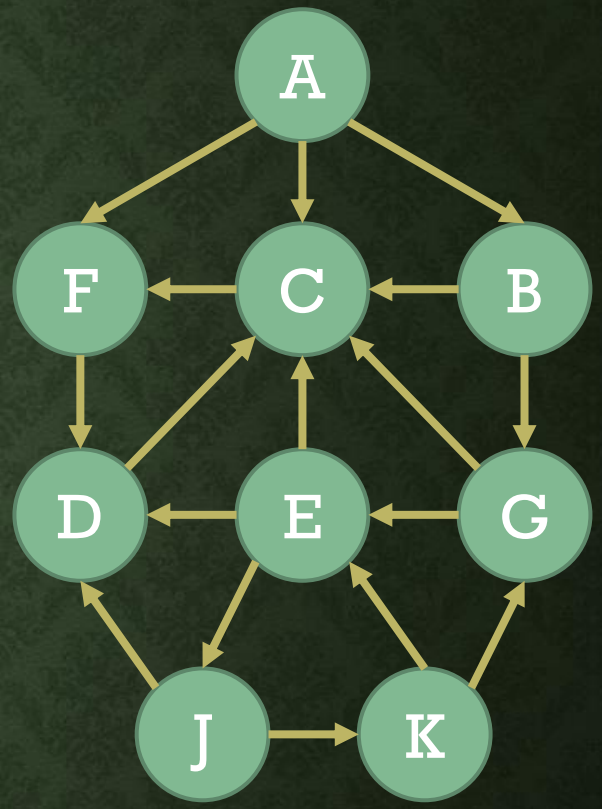
Q→ A F=1, R=0

O→ Ø

FIND A PATH FROM A TO J

- We add all neighbors of the dequeued node that have not yet been added to the queue.

Node	Neighbors	STATUS
A	B, C, F	3
B	C, G	2
C	F	2
D	C	1
E	C, D, J	1
F	D	2
G	C, E	1
J	D, K	1
K	E, G	1



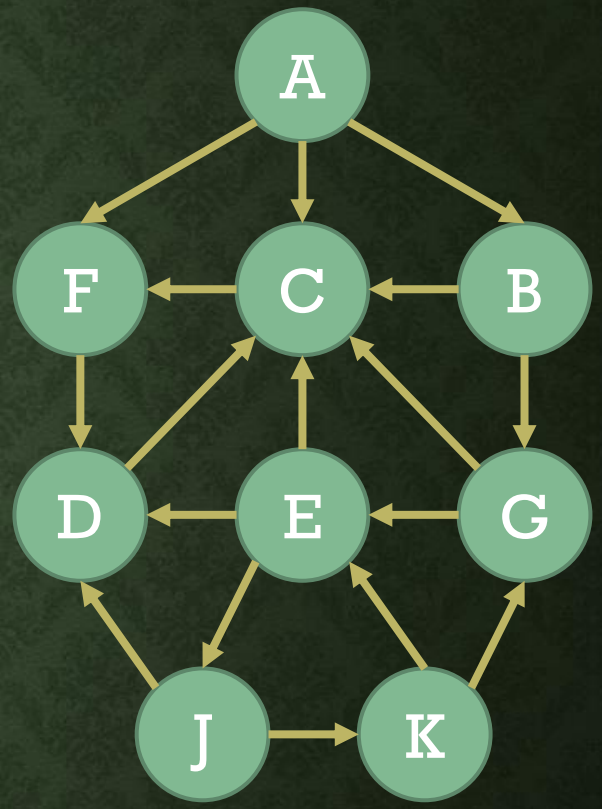
	0	1	2	3	
Q→	A	B	C	F	F=1, R=3

O→	∅	A	A	A

FIND A PATH FROM A TO J

- We visit the node at the head of the queue after dequeuing it.

Node	Neighbors	STATUS
A	B, C, F	3
B	C, G	3
C	F	2
D	C	1
E	C, D, J	1
F	D	2
G	C, E	1
J	D, K	1
K	E, G	1



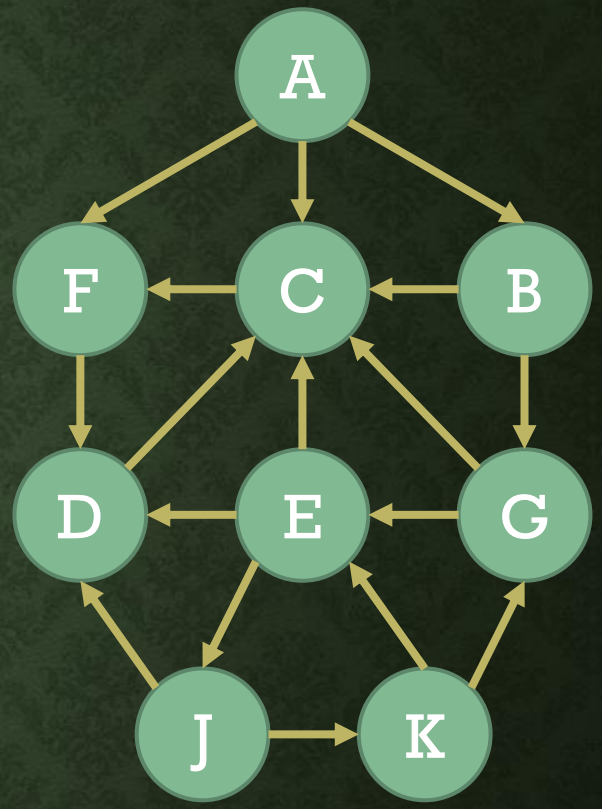
	0	1	2	3	
Q→	A	B	C	F	F=2, R=3

O→	Ø	A	A	A

FIND A PATH FROM A TO J

- We add all neighbors of the dequeued node that have not yet been added to the queue.

Node	Neighbors	STATUS
A	B, C, F	3
B	C, G	3
C	F	2
D	C	1
E	C, D, J	1
F	D	2
G	C, E	2
J	D, K	1
K	E, G	1



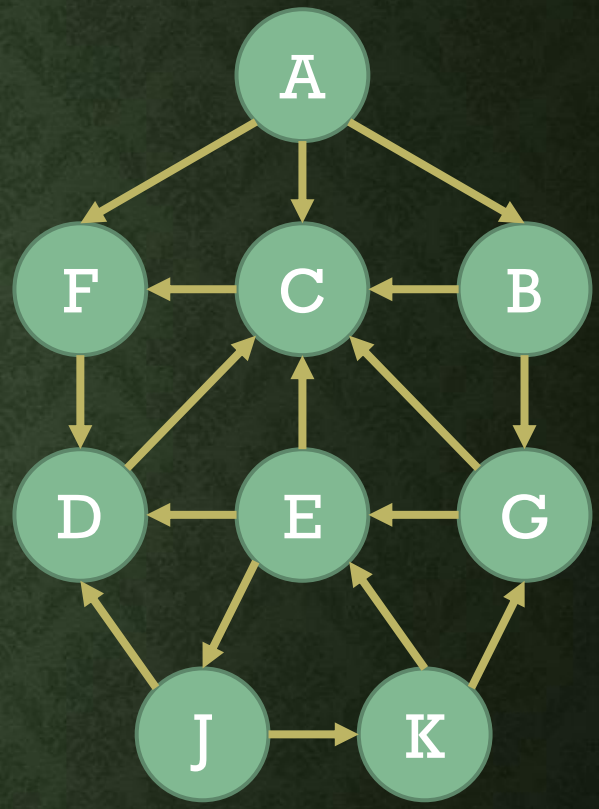
	0	1	2	3	4	
Q→	A	B	C	F	G	F=2, R=4

O→	∅	A	A	A	B

FIND A PATH FROM A TO J

- We visit the node at the head of the queue after dequeuing it.
- The dequeued node has no neighbors not yet added to the queue.

Node	Neighbors	STATUS
A	B, C, F	3
B	C, G	3
C	F	3
D	C	1
E	C, D, J	1
F	D	2
G	C, E	2
J	D, K	1
K	E, G	1



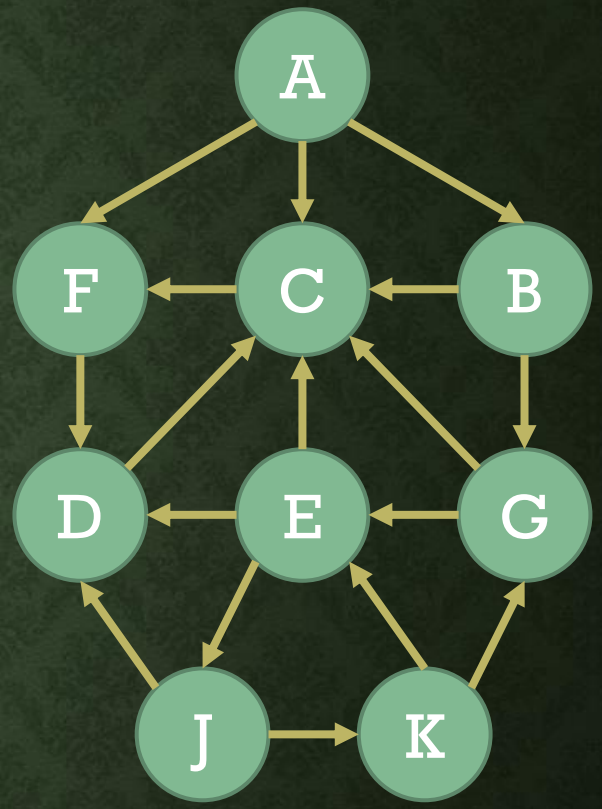
	0	1	2	3	4	
Q→	A	B	C	F	G	F=3, R=4

O→	∅	A	A	A	B

FIND A PATH FROM A TO J

- We visit the node at the head of the queue after dequeuing it.

Node	Neighbors	STATUS
A	B, C, F	3
B	C, G	3
C	F	3
D	C	1
E	C, D, J	1
F	D	3
G	C, E	2
J	D, K	1
K	E, G	1



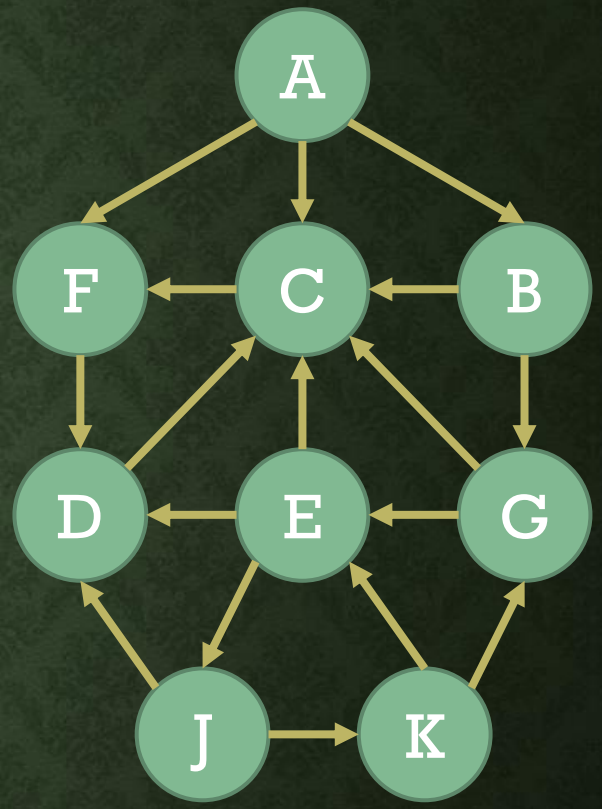
	0	1	2	3	4	
Q→	A	B	C	F	G	F=4, R=4

O→	∅	A	A	A	B

FIND A PATH FROM A TO J

- We add all neighbors of the dequeued node that have not yet been added to the queue.

Node	Neighbors	STATUS
A	B, C, F	3
B	C, G	3
C	F	3
D	C	2
E	C, D, J	1
F	D	3
G	C, E	2
J	D, K	1
K	E, G	1



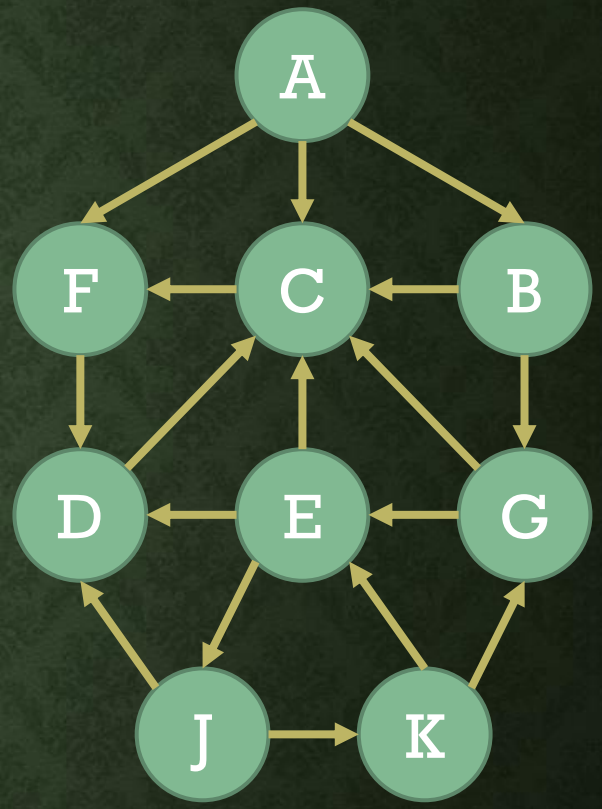
	0	1	2	3	4	5	
Q→	A	B	C	F	G	D	F=4, R=5

O→	∅	A	A	A	B	F

FIND A PATH FROM A TO J

- We visit the node at the head of the queue after dequeuing it.

Node	Neighbors	STATUS
A	B, C, F	3
B	C, G	3
C	F	3
D	C	2
E	C, D, J	1
F	D	3
G	C, E	3
J	D, K	1
K	E, G	1



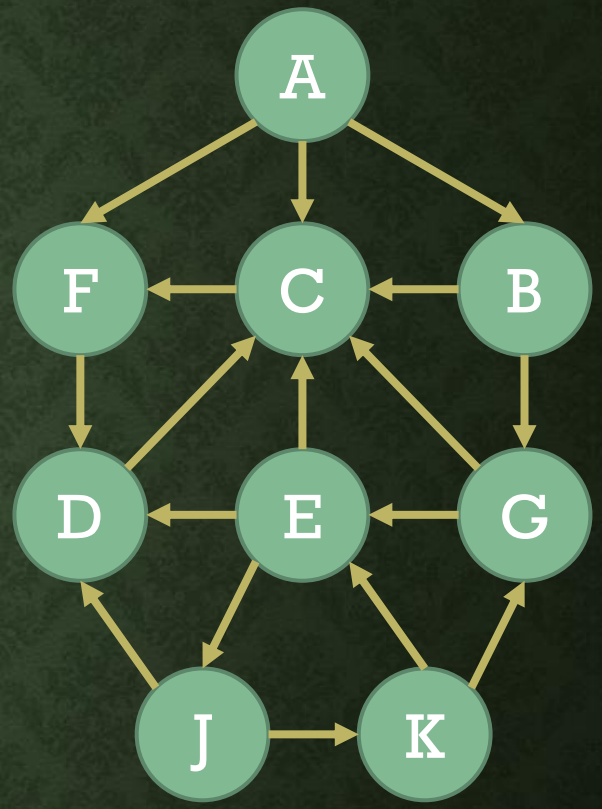
	0	1	2	3	4	5	
Q→	A	B	C	F	G	D	F=5, R=5

O→	Ø	A	A	A	B	F

FIND A PATH FROM A TO J

- We add all neighbors of the dequeued node that have not yet been added to the queue.

Node	Neighbors	STATUS
A	B, C, F	3
B	C, G	3
C	F	3
D	C	2
E	C, D, J	2
F	D	3
G	C, E	3
J	D, K	1
K	E, G	1



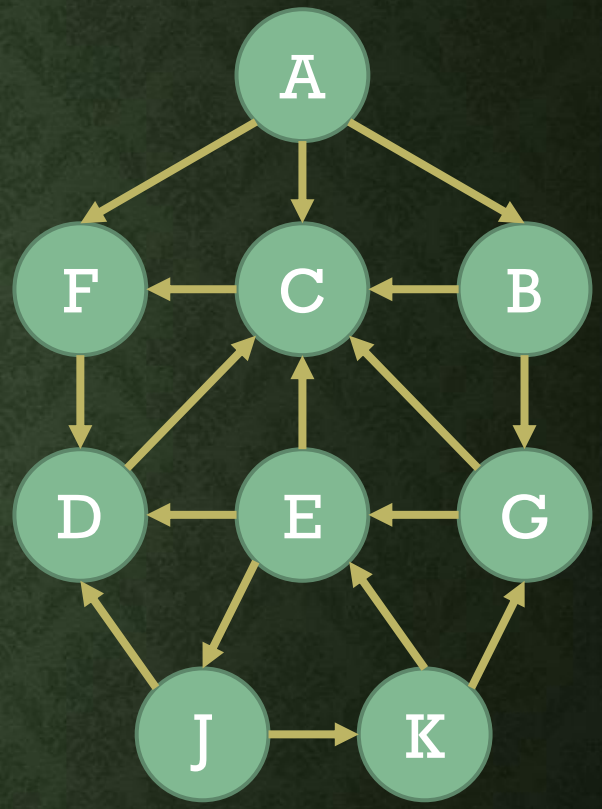
	0	1	2	3	4	5	6	
Q→	A	B	C	F	G	D	E	F=5, R=6

O→	∅	A	A	A	B	F	G

FIND A PATH FROM A TO J

- We visit the node at the head of the queue after dequeuing it.
- The dequeued node has no neighbors not yet added to the queue.

Node	Neighbors	STATUS
A	B, C, F	3
B	C, G	3
C	F	3
D	C	3
E	C, D, J	2
F	D	3
G	C, E	3
J	D, K	1
K	E, G	1



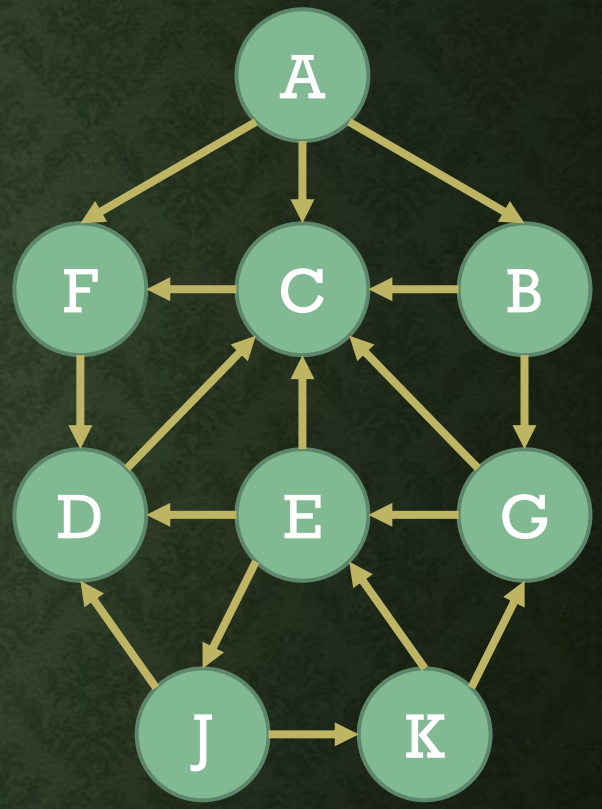
	0	1	2	3	4	5	6	
Q→	A	B	C	F	G	D	E	F=6, R=6

O→	∅	A	A	A	B	F	G

FIND A PATH FROM A TO J

- We visit the node at the head of the queue after dequeuing it.

Node	Neighbors	STATUS
A	B, C, F	3
B	C, G	3
C	F	3
D	C	3
E	C, D, J	3
F	D	3
G	C, E	3
J	D, K	1
K	E, G	1



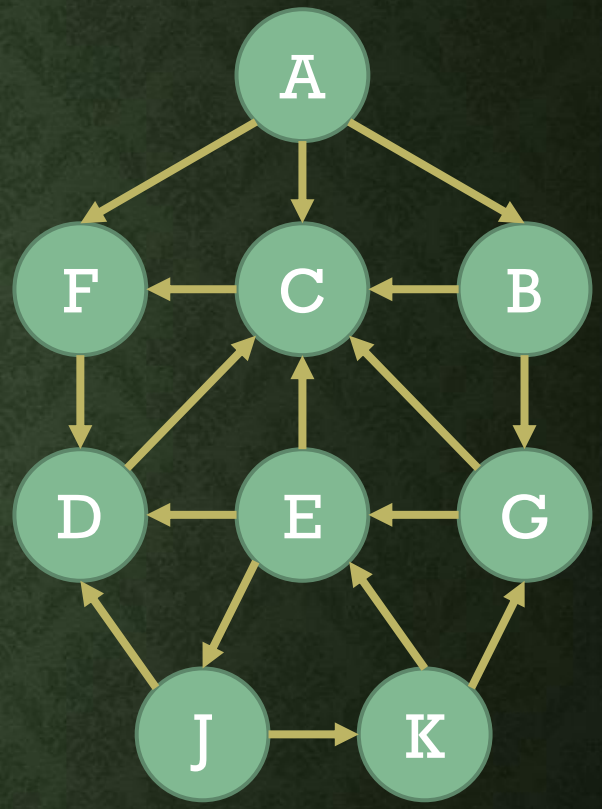
	0	1	2	3	4	5	6	
Q→	A	B	C	F	G	D	E	F=7, R=6

O→	∅	A	A	A	B	F	G

FIND A PATH FROM A TO J

- We add all neighbors of the dequeued node that have not yet been added to the queue.

Node	Neighbors	STATUS
A	B, C, F	3
B	C, G	3
C	F	3
D	C	3
E	C, D, J	3
F	D	3
G	C, E	3
J	D, K	2
K	E, G	1



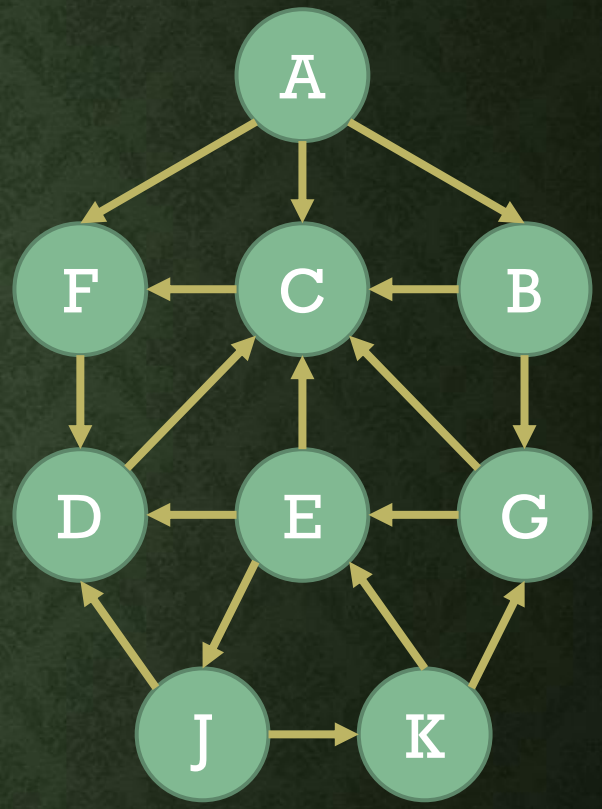
	0	1	2	3	4	5	6	7	
Q→	A	B	C	F	G	D	E	J	F=7, R=7

O→	∅	A	A	A	B	F	G	E

FIND A PATH FROM A TO J

- We visit the node at the head of the queue after dequeuing it.

Node	Neighbors	STATUS
A	B, C, F	3
B	C, G	3
C	F	3
D	C	3
E	C, D, J	3
F	D	3
G	C, E	3
J	D, K	3
K	E, G	1



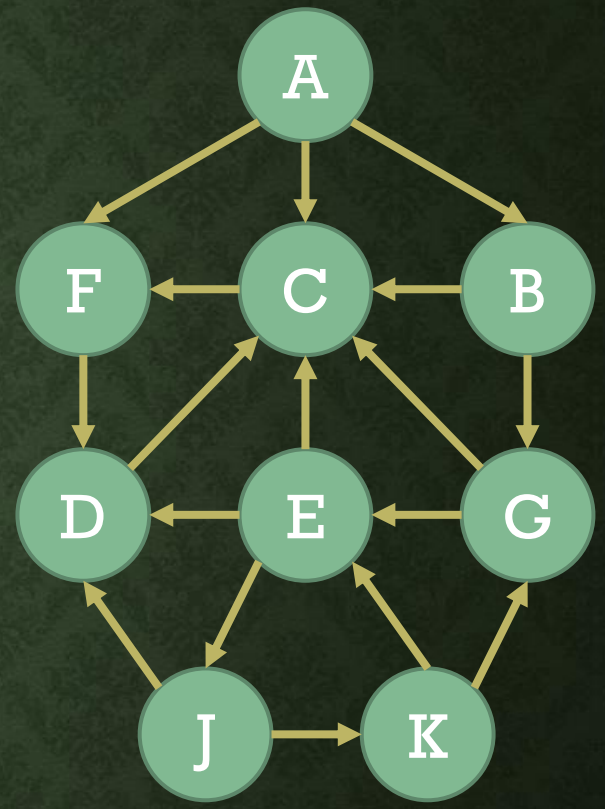
	0	1	2	3	4	5	6	7	
Q→	A	B	C	F	G	D	E	J	F=8, R=7

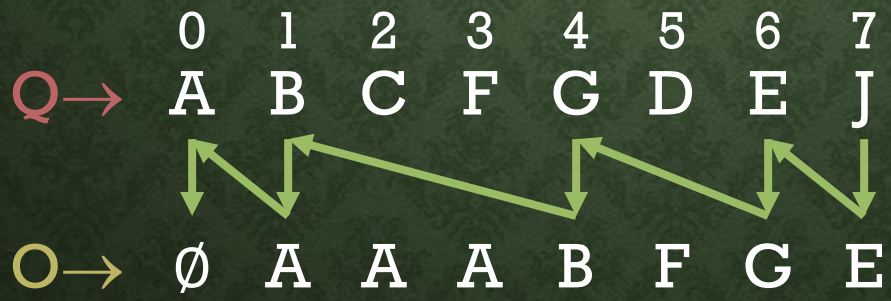
O→	∅	A	A	A	B	F	G	E

FIND A PATH FROM A TO J

- As soon as the destination is visited, we stop executing BFS.
- We now read Q and O, starting from the destination, in the manner shown here:

Node	Neighbors	STATUS
A	B, C, F	3
B	C, G	3
C	F	3
D	C	3
E	C, D, J	3
F	D	3
G	C, E	3
J	D, K	3
K	E, G	1



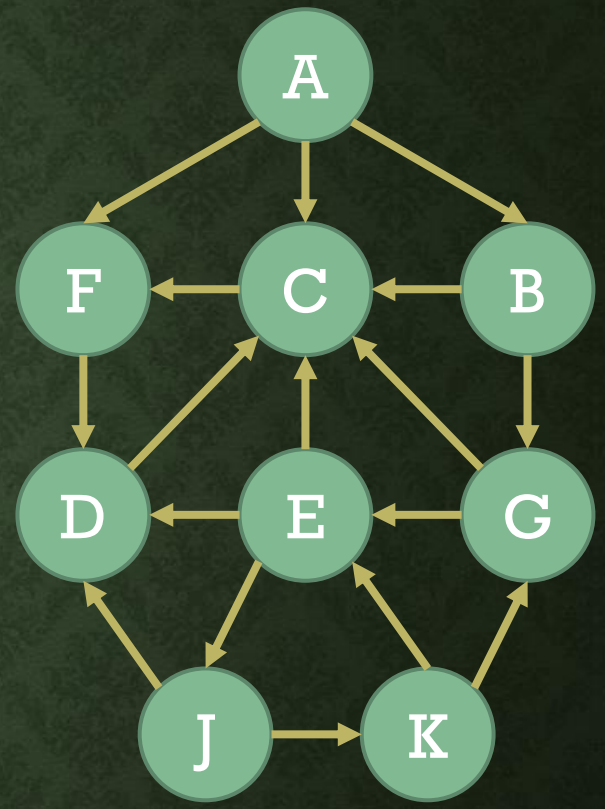


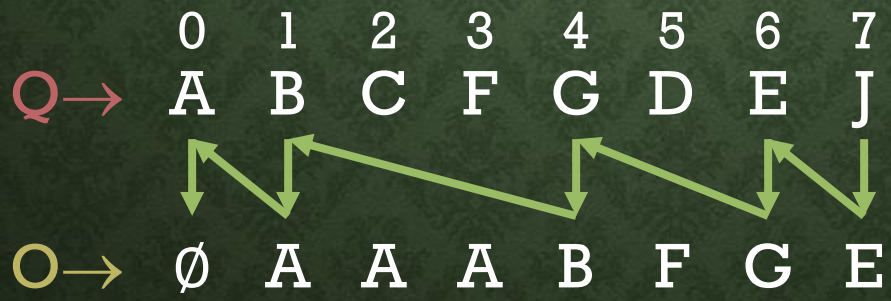
J→E→G→B→A→Ø

FIND A PATH FROM A TO J

- The path from source to destination is obtained by reversing the sequence we have found.

Node	Neighbors	STATUS
A	B, C, F	3
B	C, G	3
C	F	3
D	C	3
E	C, D, J	3
F	D	3
G	C, E	3
J	D, K	3
K	E, G	1





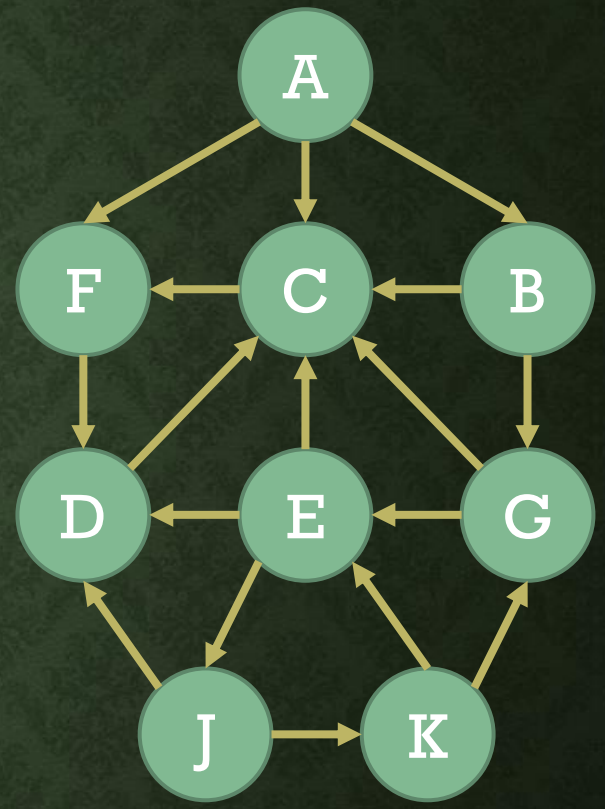
J→E→G→B→A→Ø

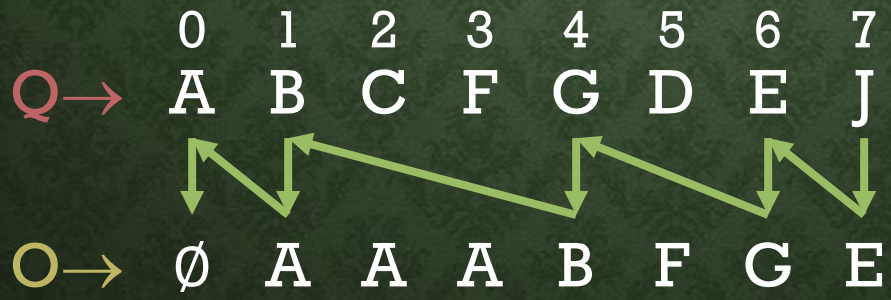
A→B→G→E→J

FIND A PATH FROM A TO J

- If no path exists between source and destination, we will finish BFS, but will not be able to enqueue the destination node.

Node	Neighbors	STATUS
A	B, C, F	3
B	C, G	3
C	F	3
D	C	3
E	C, D, J	3
F	D	3
G	C, E	3
J	D, K	3
K	E, G	1





J→E→G→B→A→Ø

A→B→G→E→J

ANALYSIS OF BFS

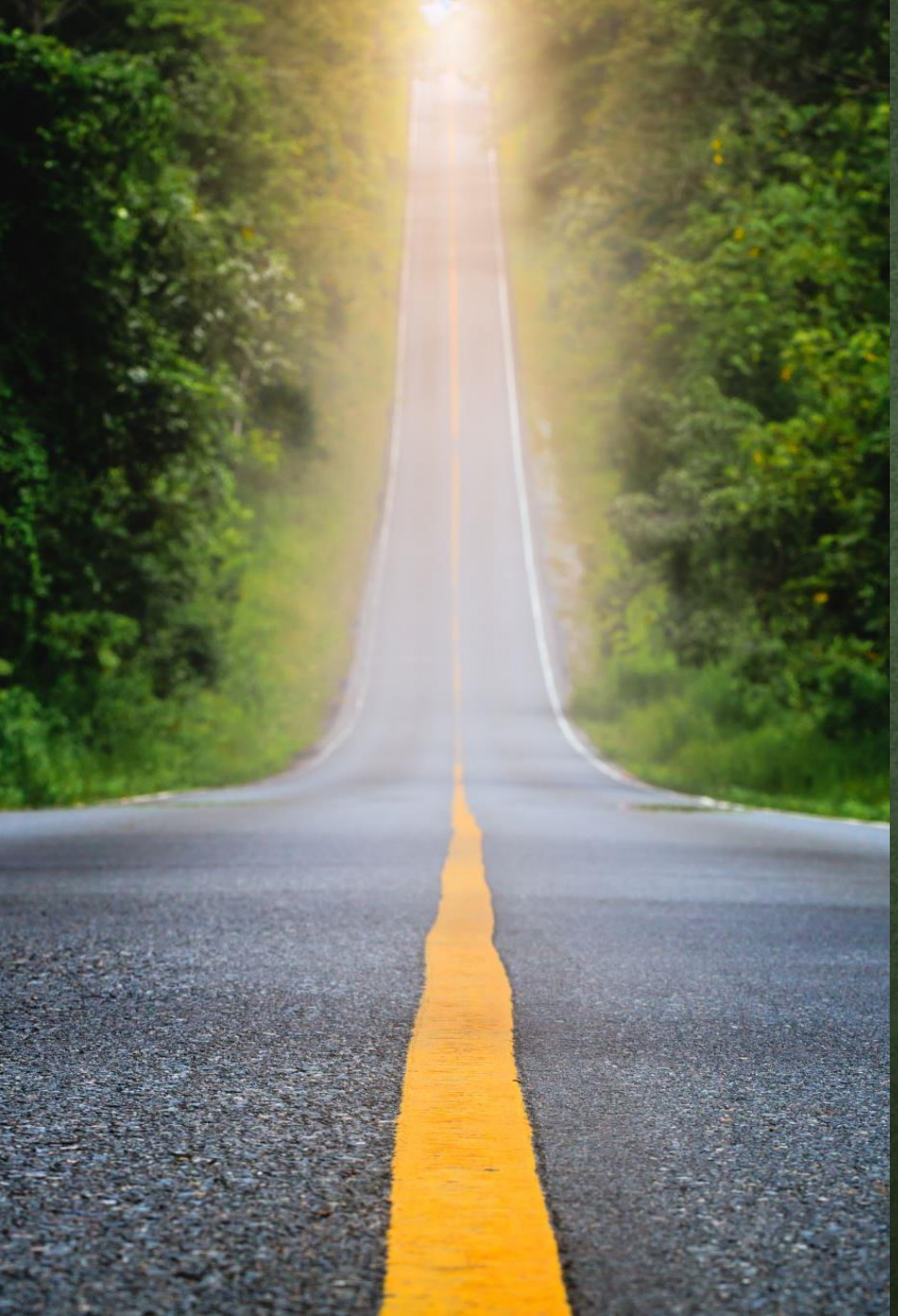
- The worst case happens when either there is no path from source to destination or the destination is the last node to be enqueued.
- In the worst case, the loop in BFS runs as many times as the number of nodes, n.
- In the body of the loop, we enqueue and dequeue, which take $O(1)$ time.
- We update the STATUS fields of neighbors of the dequeued node, but this does not take a significant amount of time, and hence can be ignored.
- BFS uses a Python list of length n to keep STATUS values of all nodes.
- BFS uses a queue implemented in a Python list, and another Python list for O.
- In the worst case, the length of both Q and O is n.

Time: $O(n)$

Space: $O(n)$

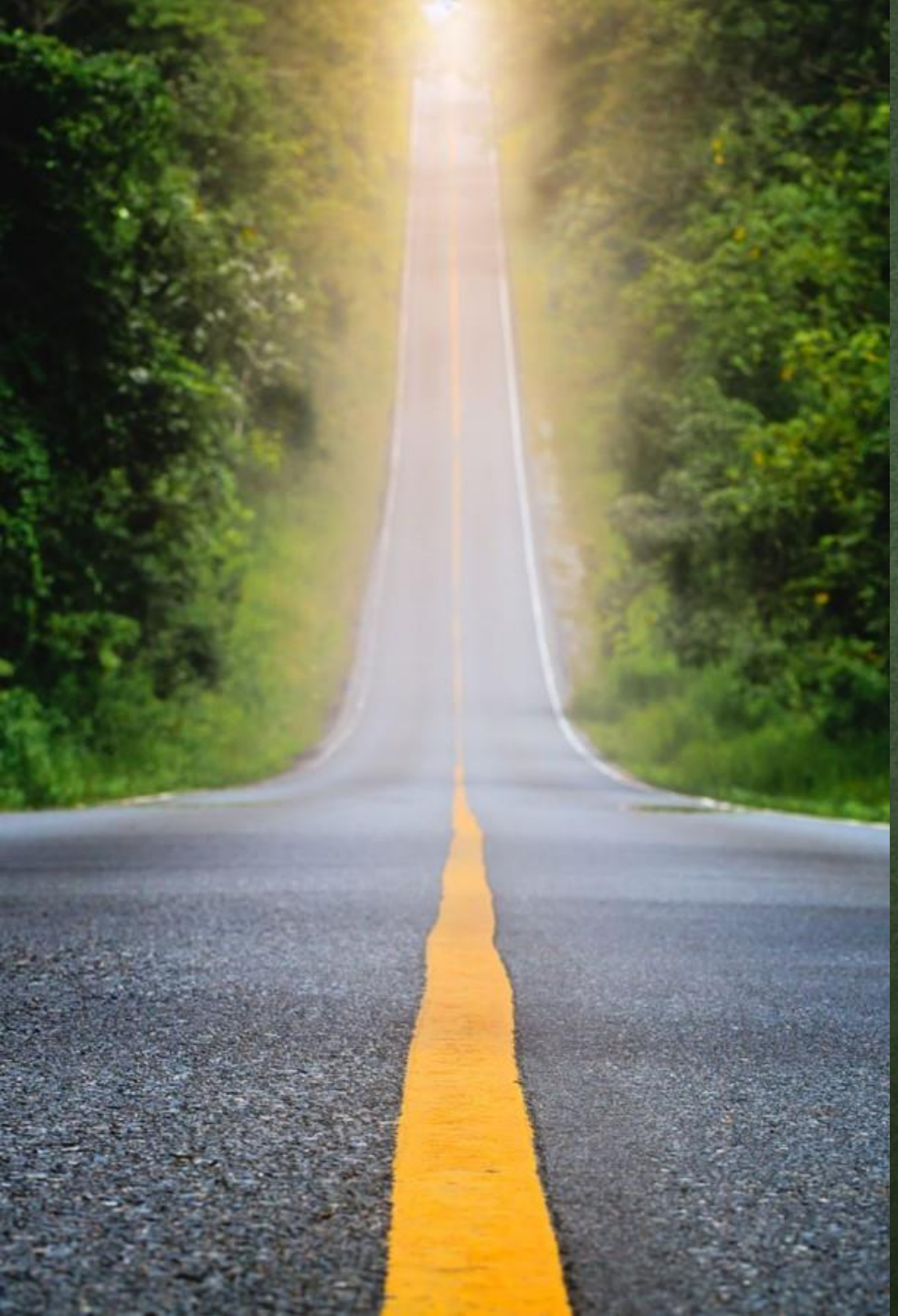
QUEUE FOR BFS

- A class qando is implemented for BFS.
- This class creates an object that has two lists, one each for Q and O. F and R variables for Q are also maintained within this object.
- A function is provided to see whether the queue in the object is empty. This is true whenever both F and R are None, or when F is greater than R.
- A function is provided to enqueue a node in the queue of the object. This function updates both Q and O, and also updates R.
- A function is provided to dequeue a node from the queue of the object. This function updates F and returns the dequeued value.
- A function is provided to read Q and O and provide the path if it exists.



DEPTH-FIRST SEARCH (DFS)

- This traversal technique uses a stack.
- Each node has a STATUS variable.
- If STATUS is 1, the node has not been added to the stack.
- If STATUS is 2, the node has been added to the stack, where it is waiting to be visited.
- If STATUS is 3, the node has been visited.



DFS ALGORITHM

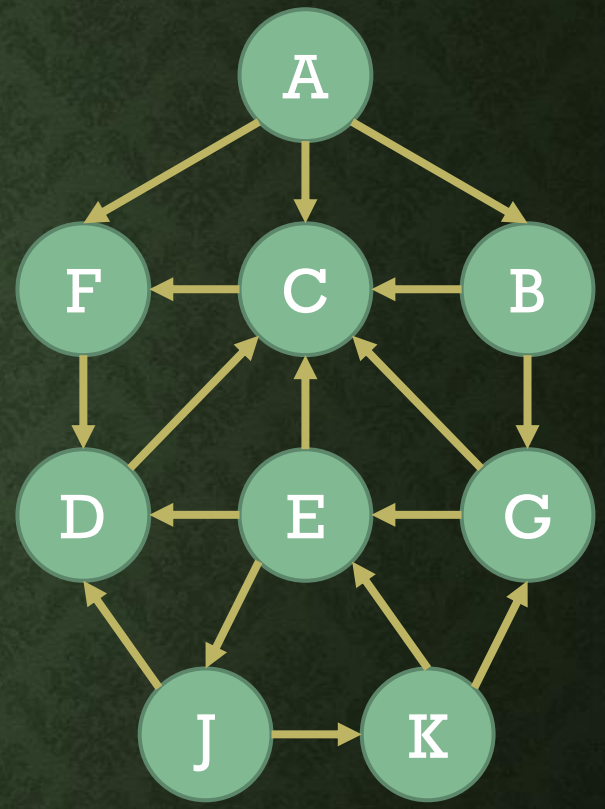
- Input: Graph G stored in memory, starting node X.

1. Set STATUS=1 for all nodes.
2. Push X in stack S. Set STATUS=2 for X.
3. Repeat until S is empty:
    a. Pop from S. Let this node be N. Visit N. Set STATUS=3 for N.
    b. Push all neighbors of N whose STATUS is 1 and set STATUS=2 for them.
4. Exit.

FIND ALL NODES REACHABLE FROM J

- We maintain the stack S.

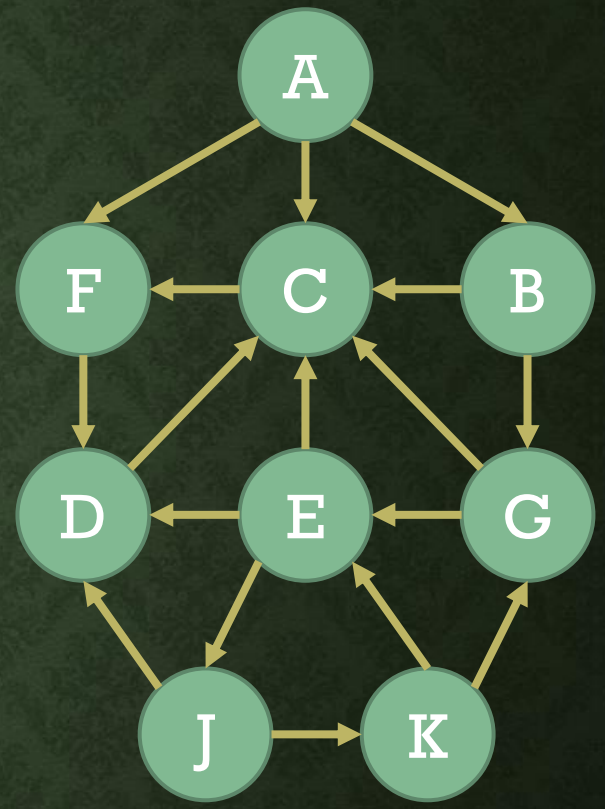
Node	Neighbors	STATUS
A	B, C, F	1
B	C, G	1
C	F	1
D	C	1
E	C, D, J	1
F	D	1
G	C, E	1
J	D, K	1
K	E, G	1



FIND ALL NODES REACHABLE FROM J

- We push the source node onto the stack.

Node	Neighbors	STATUS
A	B, C, F	1
B	C, G	1
C	F	1
D	C	1
E	C, D, J	1
F	D	1
G	C, E	1
J	D, K	2
K	E, G	1

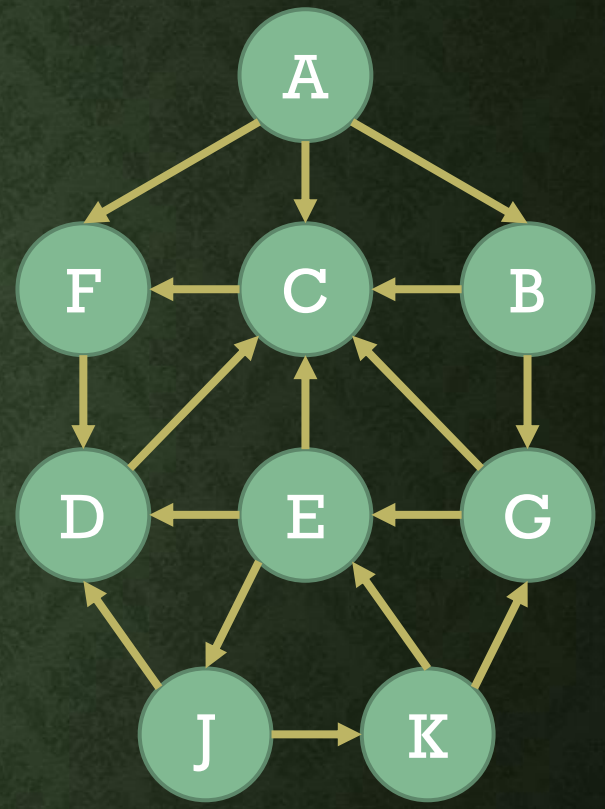


S→ J

FIND ALL NODES REACHABLE FROM J

- We visit the node at the top of the stack after popping it out.

Node	Neighbors	STATUS
A	B, C, F	1
B	C, G	1
C	F	1
D	C	1
E	C, D, J	1
F	D	1
G	C, E	1
J	D, K	3
K	E, G	1

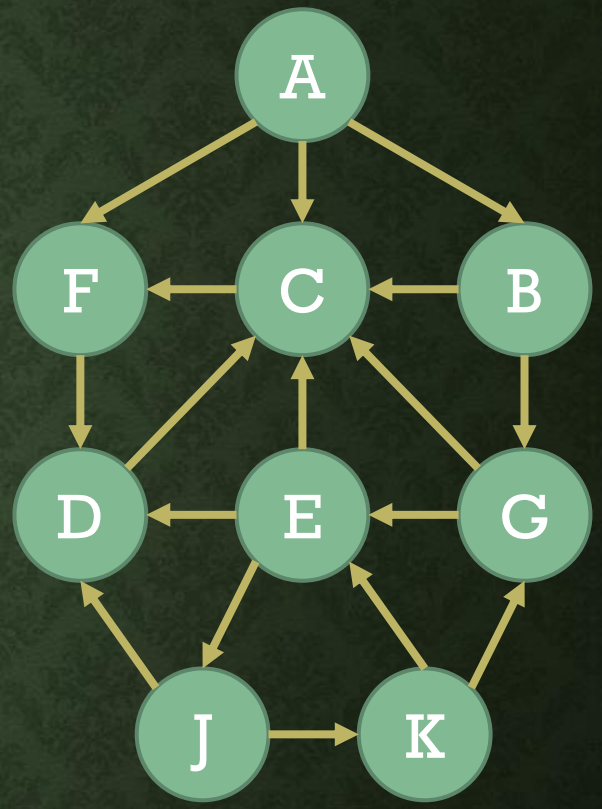


J

FIND ALL NODES REACHABLE FROM J

- We push all neighbors of the node we popped out that have not yet been added to the stack.

Node	Neighbors	STATUS
A	B, C, F	1
B	C, G	1
C	F	1
D	C	2
E	C, D, J	1
F	D	1
G	C, E	1
J	D, K	3
K	E, G	2

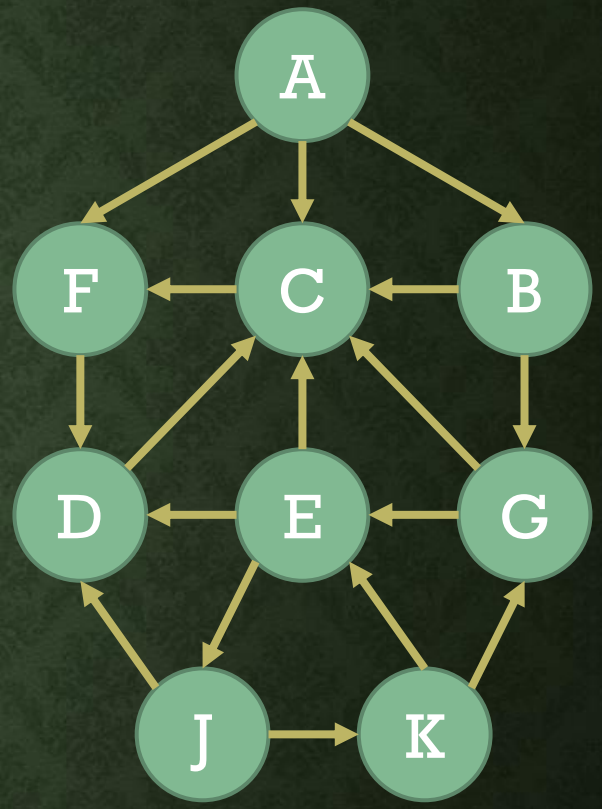


S→ D K J

FIND ALL NODES REACHABLE FROM J

- We visit the node at the top of the stack after popping it out.

Node	Neighbors	STATUS
A	B, C, F	1
B	C, G	1
C	F	1
D	C	2
E	C, D, J	1
F	D	1
G	C, E	1
J	D, K	3
K	E, G	3



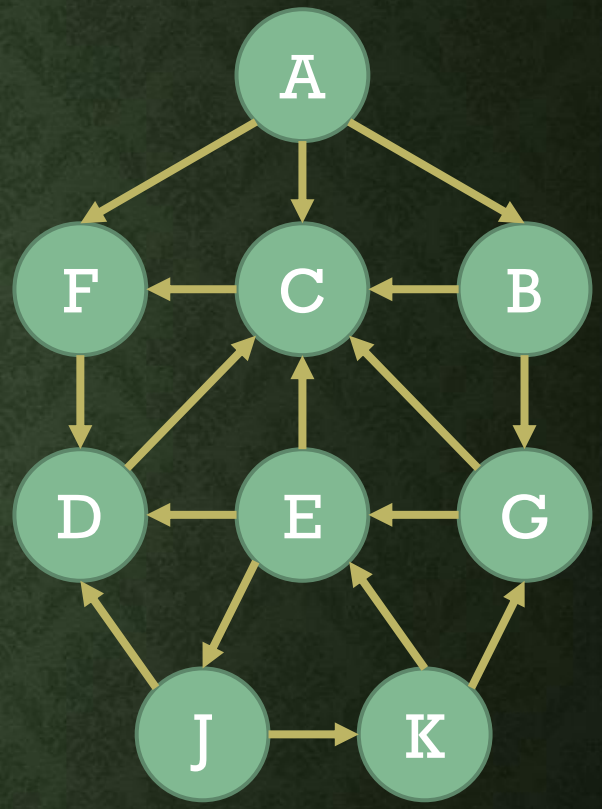
S→ D

J K

FIND ALL NODES REACHABLE FROM J

- We push all neighbors of the node we popped out that have not yet been added to the stack.

Node	Neighbors	STATUS
A	B, C, F	1
B	C, G	1
C	F	1
D	C	2
E	C, D, J	2
F	D	1
G	C, E	2
J	D, K	3
K	E, G	3



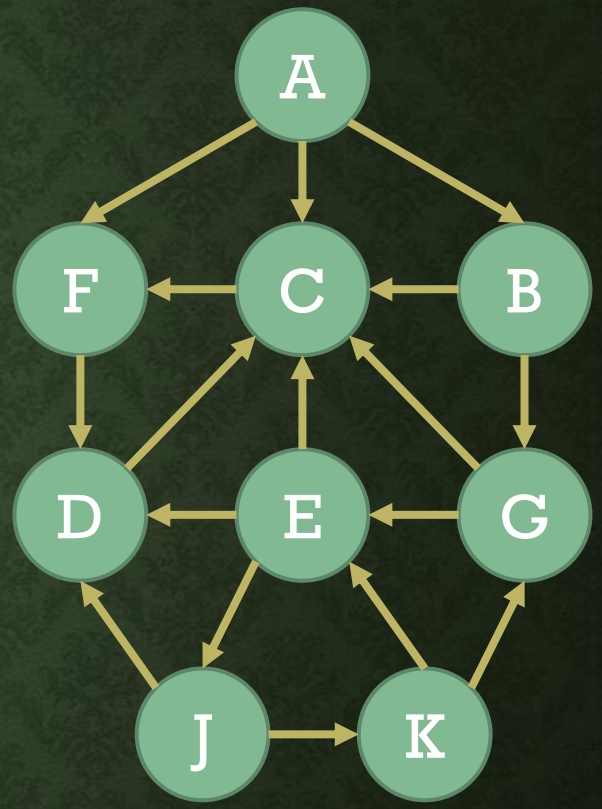
S→ D E G

J K

FIND ALL NODES REACHABLE FROM J

- We visit the node at the top of the stack after popping it out.

Node	Neighbors	STATUS
A	B, C, F	1
B	C, G	1
C	F	1
D	C	2
E	C, D, J	2
F	D	1
G	C, E	3
J	D, K	3
K	E, G	3



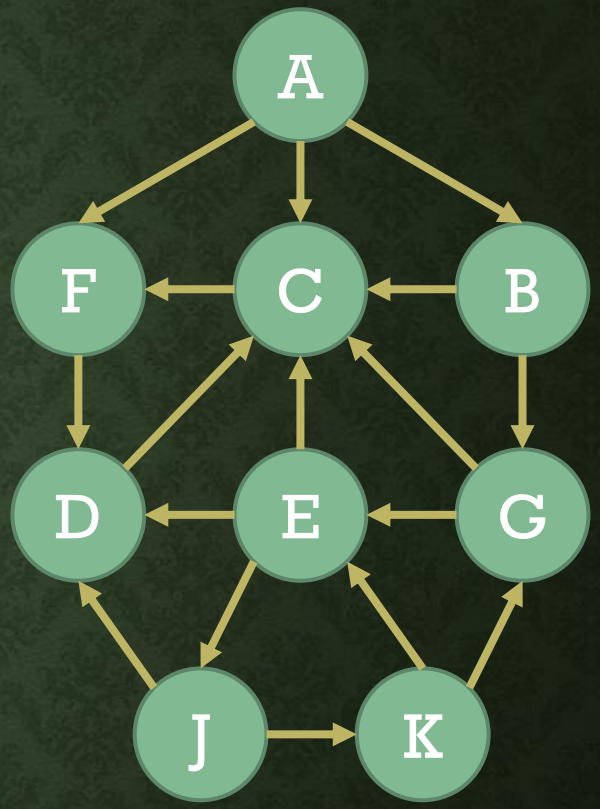
S→ D E

J K G

FIND ALL NODES REACHABLE FROM J

- We push all neighbors of the node we popped out that have not yet been added to the stack.

Node	Neighbors	STATUS
A	B, C, F	1
B	C, G	1
C	F	2
D	C	2
E	C, D, J	2
F	D	1
G	C, E	3
J	D, K	3
K	E, G	3



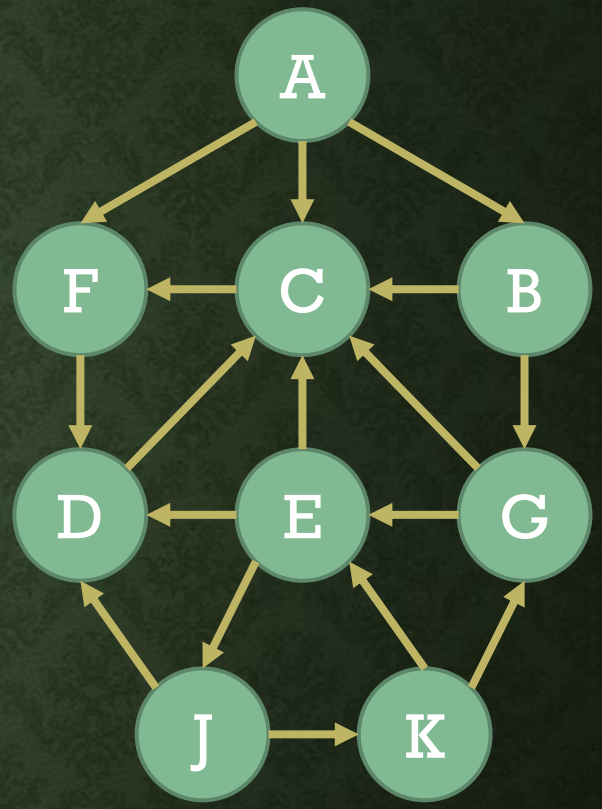
S→ D E C

J K G

FIND ALL NODES REACHABLE FROM J

- We visit the node at the top of the stack after popping it out.

Node	Neighbors	STATUS
A	B, C, F	1
B	C, G	1
C	F	3
D	C	2
E	C, D, J	2
F	D	1
G	C, E	3
J	D, K	3
K	E, G	3



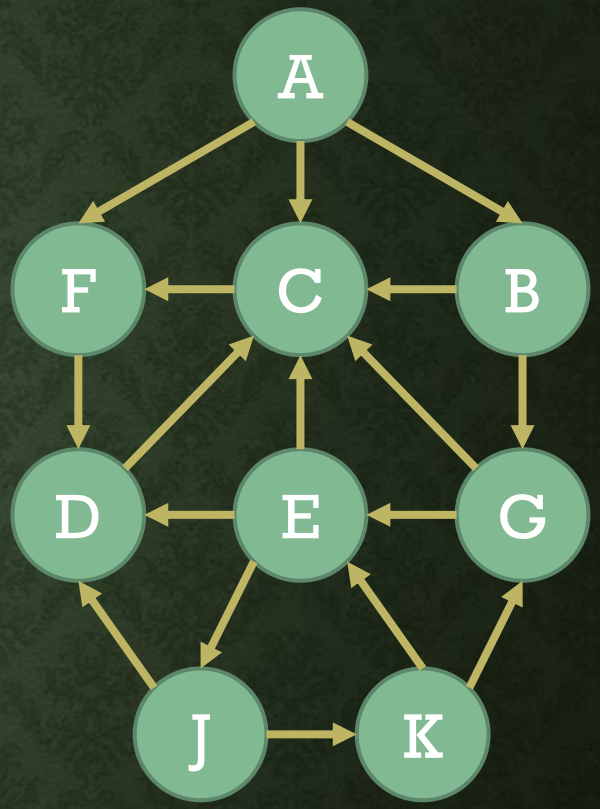
S→ D E

J K G C

FIND ALL NODES REACHABLE FROM J

- We push all neighbors of the node we popped out that have not yet been added to the stack.

Node	Neighbors	STATUS
A	B, C, F	1
B	C, G	1
C	F	3
D	C	2
E	C, D, J	2
F	D	2
G	C, E	3
J	D, K	3
K	E, G	3



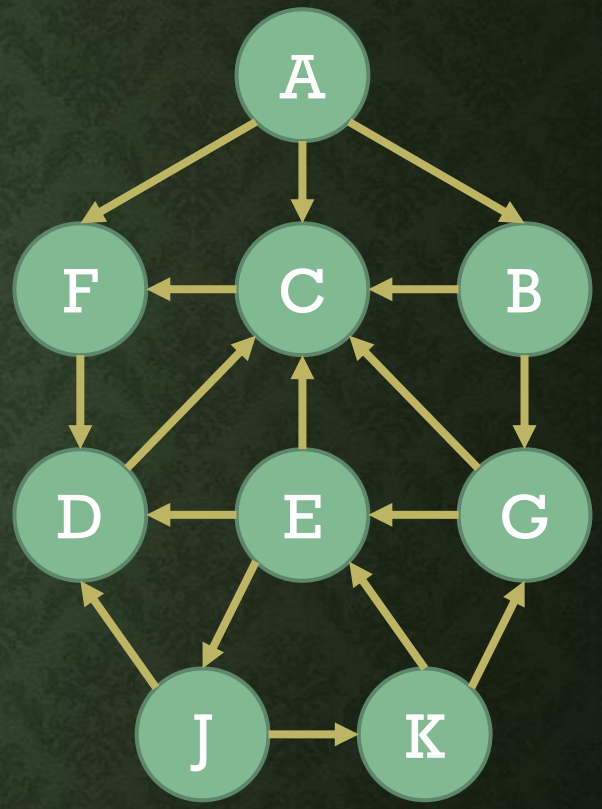
S→ D E F

J K G C

FIND ALL NODES REACHABLE FROM J

- We visit the node at the top of the stack after popping it out.
- The popped node has no neighbors not yet pushed onto the stack.

Node	Neighbors	STATUS
A	B, C, F	1
B	C, G	1
C	F	3
D	C	2
E	C, D, J	2
F	D	3
G	C, E	3
J	D, K	3
K	E, G	3



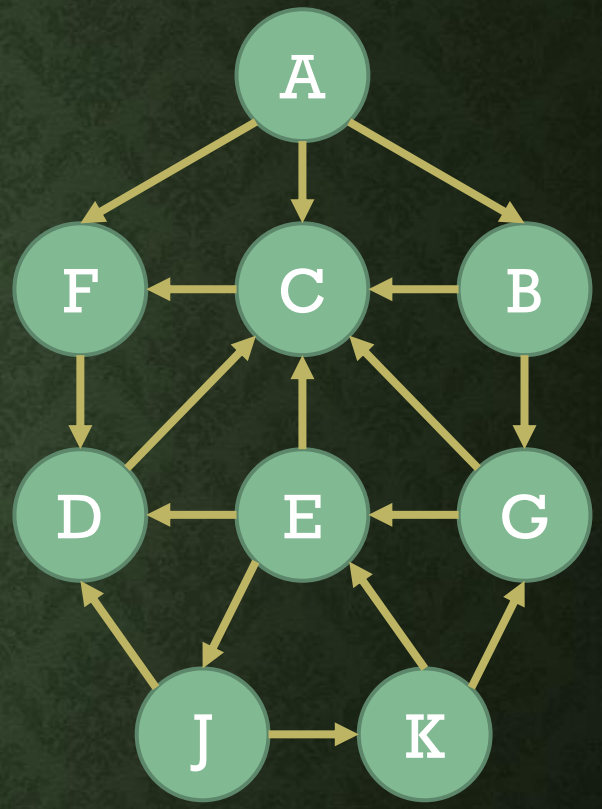
S→ D E

J K G C F

FIND ALL NODES REACHABLE FROM J

- We visit the node at the top of the stack after popping it out.
- The popped node has no neighbors not yet pushed onto the stack.

Node	Neighbors	STATUS
A	B, C, F	1
B	C, G	1
C	F	3
D	C	2
E	C, D, J	3
F	D	3
G	C, E	3
J	D, K	3
K	E, G	3



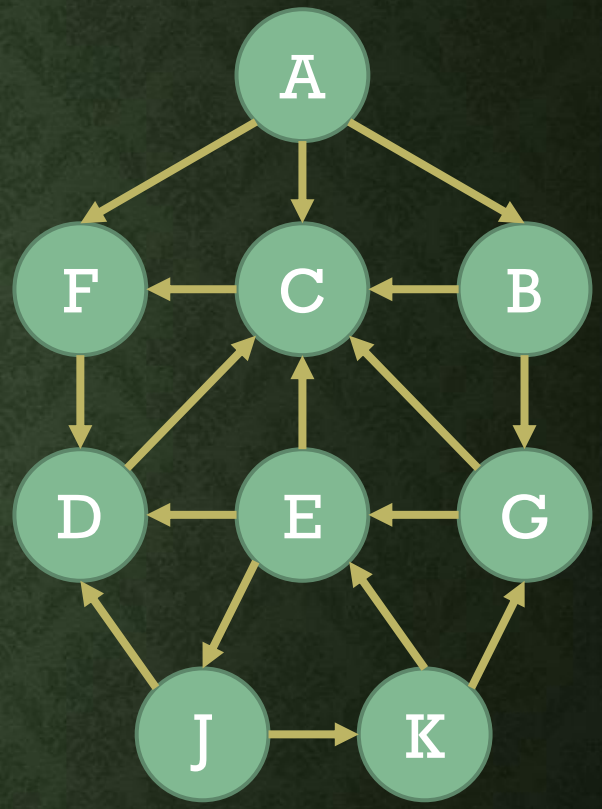
S→ D

J K G C F E

FIND ALL NODES REACHABLE FROM J

- We visit the node at the top of the stack after popping it out.
- The popped node has no neighbors not yet pushed onto the stack.

Node	Neighbors	STATUS
A	B, C, F	1
B	C, G	1
C	F	3
D	C	3
E	C, D, J	3
F	D	3
G	C, E	3
J	D, K	3
K	E, G	3

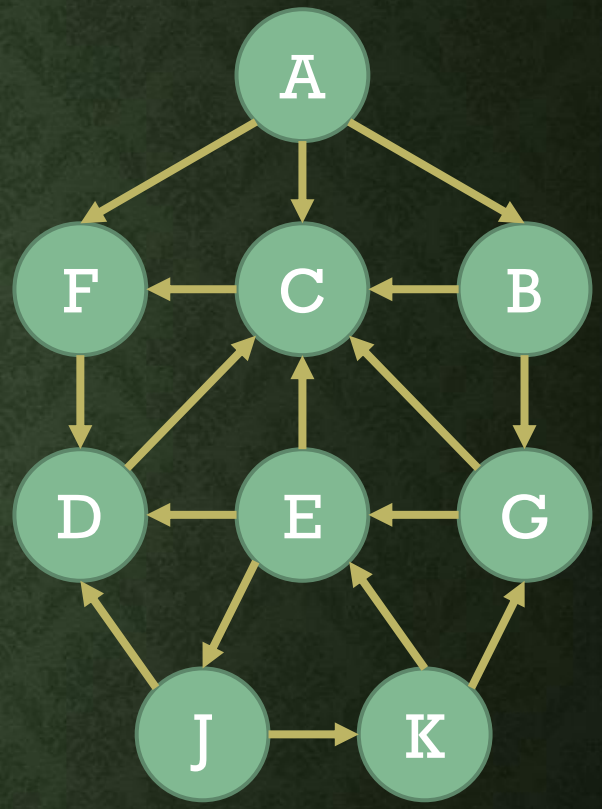


J K G C F E D

FIND ALL NODES REACHABLE FROM J

- The sequence of nodes popped out of the stack is a list of all nodes reachable from the source node.

Node	Neighbors	STATUS
A	B, C, F	1
B	C, G	1
C	F	3
D	C	3
E	C, D, J	3
F	D	3
G	C, E	3
J	D, K	3
K	E, G	3



S→

J K G C F E D

ANALYSIS OF DFS

- The worst case happens when all nodes are reachable from the source node.
- In the worst case, the loop in DFS runs as many times as the number of nodes, n.
- In the body of the loop, we push and pop, which take $O(1)$ time.
- We update the STATUS fields of neighbors of the popped node, but this does not take a significant amount of time, and hence can be ignored.
- DFS uses a Python list of length n to keep STATUS values of all nodes.
- DFS uses a stack implemented in a Python list.
- In the worst case, the length of S is n – 1.

Time: $O(n)$

Space: $O(n)$

USING PYTHON DICTIONARY TO STORE GRAPHS

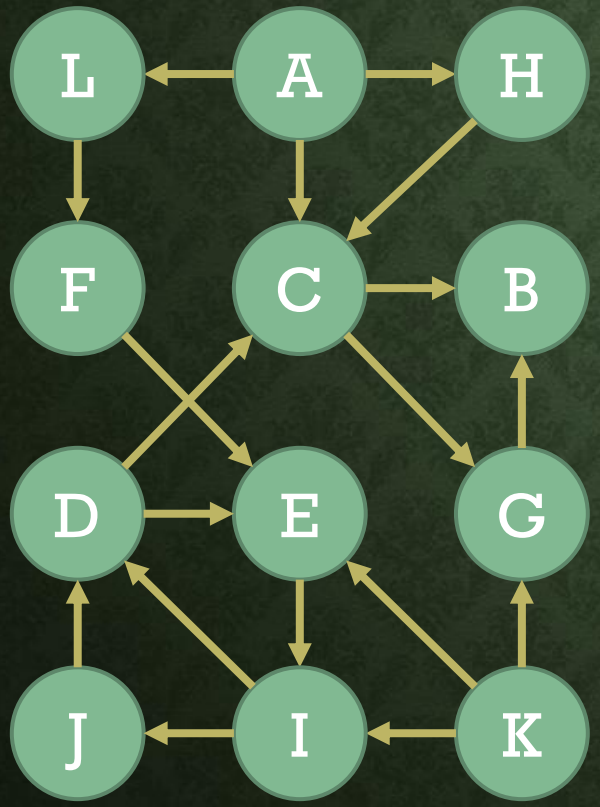
- Each node has a name.
- If we keep nodes in a list, we must search in the list to find a node by name, note its index in the list and change the value at this index in the STATUS list. This search for the node takes $O(n)$ time in the worst case.
- If we store an adjacency matrix, we must traverse the whole row for a node to note its neighbors and change their STATUS values. This traversal of the row of the adjacency matrix takes $O(n)$ time.
- The dictionary can be used to store graphs in memory.
- Dictionaries are hashed tables, a data structure that takes $O(1)$ time to search for a data item.

PROVIDING INPUT TO TRAVERSAL ALGORITHMS

```
nodes = {'A':0, 'B':1, 'C':2, 'D':3, 'E':4, 'F':5, 'G':6,'J':7, 'K':8}

adjlist = {'A':'BCF', 'B':'CG', 'C':'F', 'D':'C', 'E':'CDJ', 'F':'D', 'G':'CE','J':'DK',\
        'K':'EG'}
```

HOMEWORK



Find a path from A to I.

Find all nodes reachable from A.

WHAT DID WE LEARN TODAY?

We explored algorithms for graph traversal.

We looked at data structures that facilitate graph traversal algorithms.

We investigated time and space complexities of graph traversal algorithms.

THINGS TO DO

Read the book!

Note your questions and put them up in the relevant online session.

Email suggestions on content or quality of this lecture at uroojain@neduet.edu.pk